

SQL 프로그래밍

스토어드 프로시저 제어문과 동적 SQL

IF · CASE · WHILE · PREPARE / EXECUTE

코드를 "저장해 두고" 필요할 때 CALL로 실행한다.

```
CREATE PROCEDURE ...  
BEGIN  
  IF ... THEN  
  CASE ...  
  WHILE ... DO  
  PREPARE ...  
END
```

오늘 배울 핵심 흐름

프로시저 안에서 조건, 선택, 반복, 동적 실행을 사용하는 방법을 익힙니다.



구문	역할	오늘 코드에서 보는 예
IF	조건이 참인지 거짓인지 판단	ifProc1, ifProc2, ifProc3
CASE	여러 조건 중 하나를 선택	caseProc, 회원등급 분류
WHILE	조건이 참인 동안 반복	whileProc, whileProc2
PREPARE / EXECUTE	문자열 SQL을 실행 가능한 문장으로 변환	myQuery, gate_table 입력

프로시저 기본 구조와 IF 문

DELIMITER를 바꾸는 이유: 프로시저 내부의 세미콜론과 전체 종료 기호를 구분하기 위해서입니다.

```
DROP PROCEDURE IF EXISTS ifProc1; -- 기존에 만든적이 있다면 삭제
DELIMITER $$
CREATE PROCEDURE ifProc1()
BEGIN
    IF 100 = 100 THEN
        SELECT '100은 100과 같습니다.';
    END IF;
END $$
DELIMITER ;
CALL ifProc1();
```

수업 포인트

- 1) DROP PROCEDURE IF EXISTS: 같은 이름이 있으면 먼저 삭제
- 2) CREATE PROCEDURE: 프로시저 생성
- 3) BEGIN ~ END: 실행할 SQL 블록
- 4) CALL: 저장된 프로시저 실행

IF 기본형

```
IF 조건 THEN
    실행문;
END IF;
```

조건이 참이면 THEN 아래 문장이 실행됩니다.

IF ELSE: 참일 때와 거짓일 때를 나누기

DECLARE로 변수를 만들고, SET으로 값을 넣은 뒤 조건을 판단합니다.

```
DROP PROCEDURE IF EXISTS ifProc2;
DELIMITER $$
CREATE PROCEDURE ifProc2()
BEGIN
    DECLARE myNum INT; -- myNum 변수선언
    SET myNum = 200; -- 변수에 값 대입
    IF myNum = 100 THEN
        SELECT '100입니다.';
    ELSE
        SELECT '100이 아닙니다.';
    END IF;
END $$
DELIMITER ;
CALL ifProc2();
```

변수 사용 순서

DECLARE myNum INT;
→ 정수형 변수 선언

SET myNum = 200;
→ 변수에 값 대입

결과 예측

myNum은 200입니다.
따라서 myNum = 100 조건은 거짓입니다.
실행 결과는 ELSE 부분입니다.

IF + SELECT INTO: 테이블 값을 변수에 넣기 ①

member 테이블에서 특정 회원의 debut_date를 가져와 변수에 저장합니다.

```
DROP PROCEDURE IF EXISTS ifProc3;
DELIMITER $$
CREATE PROCEDURE ifProc3()
BEGIN
    DECLARE debutDate DATE; -- 데뷔일
    DECLARE curDate DATE; -- 오늘
    DECLARE days INT; -- 활동한 일수

    SELECT debut_date INTO debutDate -- debut_date 결과를 hireDATE에 대입
    FROM market_db.member
    WHERE mem_id = 'APN';
```

SELECT ... INTO

조회 결과 1개를 변수에 저장할 때 사용합니다.

```
SELECT 컬럼 INTO 변수
FROM 테이블
WHERE 조건;
```

주의

조회 결과가 여러 행이면 오류가 날 수 있습니다.
조건을 정확히 주어 1행만 나오게 해야 합니다.

IF + 날짜 함수: 활동 일수 계산하기 ②

CURRENT_DATE()와 DATEDIFF()로 날짜 차이를 구한 뒤 조건문으로 메시지를 나눕니다.

```
SET curDATE = CURRENT_DATE(); -- 현재 날짜
SET days = DATEDIFF(curDATE, debutDate); -- 날짜의 차이, 일 단위

IF (days/365) >= 5 THEN -- 5년이 지났다면
    SELECT CONCAT('데뷔한지 ', days, '일이나 지났습니다. 핑순이들 축하합니다!');
ELSE
    SELECT '데뷔한지 ' + days + '일밖에 안되었네요. 핑순이들 화이팅~' ;
END IF;
END $$
DELIMITER ;
CALL ifProc3();

SELECT CURRENT_DATE(), DATEDIFF('2021-12-31', '2000-1-1');
```

DATEDIFF()

DATEDIFF(나중날짜, 이전날짜)
두 날짜 사이의 차이를 "일수"로 반환합니다.

문자열 결합 주의

MySQL에서 문자열을 이어 붙일 때는
CONCAT()을 쓰는 것이 안전합니다.

+는 숫자 연산자로 해석될 수 있습니다.

CASE 문: 여러 조건 중 하나 고르기

점수 범위에 따라 학점을 A~F로 결정하는 절차적 CASE 문입니다.

```
DROP PROCEDURE IF EXISTS caseProc;
DELIMITER $$
CREATE PROCEDURE caseProc()
BEGIN
    DECLARE point INT ;
    DECLARE credit CHAR(1);
    SET point = 88 ;

    CASE
        WHEN point >= 90 THEN
            SET credit = 'A';
        WHEN point >= 80 THEN
            SET credit = 'B';
        WHEN point >= 70 THEN
            SET credit = 'C';
        WHEN point >= 60 THEN
            SET credit = 'D';
        ELSE
            SET credit = 'F';
    END CASE;
    SELECT CONCAT('취득점수==>', point), CONCAT('학점==>', credit);
END $$
DELIMITER ;
CALL caseProc();
```

조건	credit
point >= 90	A
point >= 80	B
point >= 70	C
point >= 60	D
그 외	F

왜 88점은 B인가?

CASE는 위에서부터 조건을 검사합니다.
88은 90 이상은 아니지만 80 이상이므로 B가 됩니다.

IF와 CASE의 선택 기준

둘 다 조건을 처리하지만, 수업에서는 “조건의 모양”에 따라 선택하면 이해하기 쉽습니다.

상황	추천	예시
참/거짓 2갈래	IF / IF ELSE	100인가? 5년 이상인가?
점수 구간처럼 여러 갈래	CASE	90 이상 A, 80 이상 B ...
SELECT 결과에 따라 값을 붙이기	CASE 표현식	회원등급 만들기



핵심: CASE는 “위에서 아래로” 검사합니다. 먼저 참이 되는 WHEN에서 결정됩니다.

회원별 총구매액 계산: GROUP BY와 ORDER BY

CASE로 회원등급을 만들기 전에, 먼저 회원별 총구매액을 계산합니다.

```
SELECT mem_id, SUM(price*amount) "총구매액"  
FROM buy  
GROUP BY mem_id;  
  
SELECT mem_id, SUM(price*amount) "총구매액"  
FROM buy  
GROUP BY mem_id  
ORDER BY SUM(price*amount) DESC ;
```

SUM(price*amount)

가격 × 수량을 모두 더해 총구매액을 만듭니다.

GROUP BY mem_id

같은 회원 아이디끼리 묶어 계산합니다.

ORDER BY ... DESC

총구매액이 큰 회원부터 내림차순 정렬합니다.

JOIN으로 회원 이름 함께 보기

구매 테이블에는 mem_id만 있으므로 member 테이블과 연결해 mem_name을 가져옵니다.

```
SELECT B.mem_id, M.mem_name, SUM(price*amount) "총구매액"  
FROM buy B  
      INNER JOIN member M  
      ON B.mem_id = M.mem_id  
GROUP BY B.mem_id  
ORDER BY SUM(price*amount) DESC ;
```

```
SELECT M.mem_id, M.mem_name, SUM(price*amount) "총구매액"  
FROM buy B  
      RIGHT OUTER JOIN member M  
      ON B.mem_id = M.mem_id  
GROUP BY M.mem_id  
ORDER BY SUM(price*amount) DESC ;
```

INNER JOIN

구매 기록이 있는 회원만 보입니다. buy와 member 양쪽에 모두 있는 mem_id만 결과에 포함됩니다.

RIGHT OUTER JOIN

오른쪽 테이블 member를 기준으로, 구매하지 않은 회원도 포함합니다. 총구매액이 NULL이 될 수 있습니다.

SELECT 안의 CASE: 총구매액으로 회원등급 만들기

집계 결과를 기준으로 "최우수·우수·일반·유령고객"으로 분류합니다.

```
SELECT M.mem_id, M.mem_name, SUM(price*amount) "총구매액",  
       CASE  
         WHEN (SUM(price*amount) >= 1500) THEN '최우수고객'  
         WHEN (SUM(price*amount) >= 1000) THEN '우수고객'  
         WHEN (SUM(price*amount) >= 1 ) THEN '일반고객'  
         ELSE '유령고객'  
       END "회원등급"  
FROM buy B  
     RIGHT OUTER JOIN member M  
     ON B.mem_id = M.mem_id  
GROUP BY M.mem_id  
ORDER BY SUM(price*amount) DESC ;
```

총구매액	회원등급
1500 이상	최우수고객
1000 이상	우수고객
1 이상	일반고객
그 외	유령고객

핵심 차이

프로시저 안의 CASE는 END CASE;
SELECT 안의 CASE 표현식은 END "별칭" 형
태로 결과 열을 만듭니다.

WHILE 문: 조건이 참인 동안 반복하기

1부터 100까지 i를 증가시키며 hap에 누적합니다.

```
DROP PROCEDURE IF EXISTS whileProc;
DELIMITER $$
CREATE PROCEDURE whileProc()
BEGIN
    DECLARE i INT; -- 1에서 100까지 증가할 변수
    DECLARE hap INT; -- 더한 값을 누적할 변수
    SET i = 1;
    SET hap = 0;

    WHILE (i <= 100) DO
        SET hap = hap + i; -- hap의 원래의 값에 i를 더해서 다시 hap에 넣으라는 의미
        SET i = i + 1; -- i의 원래의 값에 1을 더해서 다시 i에 넣으라는 의미
    END WHILE;

    SELECT '1부터 100까지의 합 ==>', hap;
END $$
DELIMITER ;
CALL whileProc();
```



반복문의 핵심

반복을 멈출 수 있도록 $i = i + 1$ 같은 변화가 반드시 있어야 합니다. 없으면 무한 반복이 될 수 있습니다.

WHILE + label: 반복문에 이름 붙이기 ①

label을 붙이면 ITERATE와 LEAVE가 어느 반복문을 가리키는지 명확해집니다.

```
DROP PROCEDURE IF EXISTS whileProc2;
DELIMITER $$
CREATE PROCEDURE whileProc2()
BEGIN
    DECLARE i INT; -- 1에서 100까지 증가할 변수
    DECLARE hap INT; -- 더한 값을 누적할 변수
    SET i = 1;
    SET hap = 0;

    myWhile:
    WHILE (i <= 100) DO -- While문에 label을 지정
        IF (i%4 = 0) THEN
            SET i = i + 1;
            ITERATE myWhile; -- 지정한 label문으로 가서 계속 진행
        END IF;
    END WHILE;
```

ITERATE

현재 반복의 나머지를 건너뛰고, 지정한 label의 WHILE 조건 검사로 돌아갑니다.

i%4 = 0

i를 4로 나눈 나머지가 0이면 4의 배수입니다.
따라서 4, 8, 12 ...는 합계에서 제외됩니다.

WHILE + LEAVE: 조건을 만족하면 반복 종료 ②

hap이 1000을 넘는 순간 LEAVE로 반복문을 빠져나옵니다.

```
SET hap = hap + i;
IF (hap > 1000) THEN
    LEAVE myWhile; --指定的 label문을 떠남. 즉, While 종료.
END IF;
SET i = i + 1;
END WHILE;

SELECT '1부터 100까지의 합(4의 배수 제외), 1000 넘으면 종료 ==>', hap;
END $$
DELIMITER ;
CALL whileProc2();
```

LEAVE

지정한 label의 반복문을 종료합니다.
여기서는 myWhile 반복문을 즉시 빠져나갑니다.

4의 배수?
건너뛰기

합계 누적

hap > 1000?
종료

아니면
i 증가

동적 SQL ①: 문자열을 SQL 문으로 실행하기

PREPARE는 문자열 SQL을 준비하고, EXECUTE는 준비된 SQL을 실행합니다.

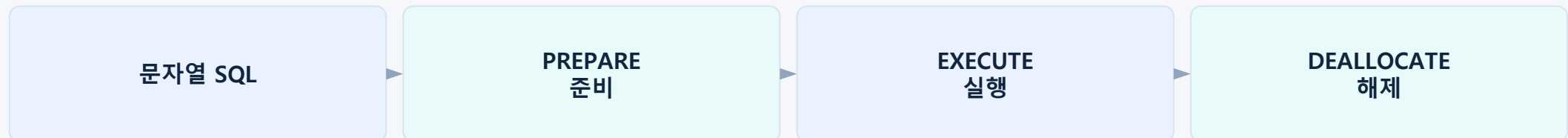
```
use market_db;
PREPARE myQuery FROM 'SELECT * FROM member WHERE mem_id =
"BLK"';
EXECUTE myQuery;
DEALLOCATE PREPARE myQuery;
```

언제 쓰나?

테이블명, 조건, 값 등이 실행 시점에 결정되어 SQL 문장을 동적으로 만들어야 할 때 사용합니다.

주의

동적 SQL은 강력하지만, 사용자 입력을 그대로 붙이면 보안 문제가 생길 수 있습니다.



동적 SQL ②: ? 자리표시자와 EXECUTE ... USING

현재 시각을 변수에 저장한 뒤, 준비된 INSERT 문에 값으로 전달합니다.

```
DROP TABLE IF EXISTS gate_table;
CREATE TABLE gate_table (id INT AUTO_INCREMENT PRIMARY KEY, entry_time DATETIME);

SET @curDate = CURRENT_TIMESTAMP(); -- 현재 날짜와 시간

PREPARE myQuery FROM 'INSERT INTO gate_table VALUES(NULL, ?)';
EXECUTE myQuery USING @curDate;
DEALLOCATE PREPARE myQuery;

SELECT * FROM gate_table;
```

?

실행할 때 값이 들어갈 자리입니다.
SQL 문장을 미리 준비해 두고 값은 나중에 전달합니다.

@curDate

사용자 변수입니다.
CURRENT_TIMESTAMP()의 결과를 저장하고 EXECUTE ... USING으로 전달합니다.

결과 확인

```
SELECT * FROM gate_table;
```

입력된 현재 시각을 확인합니다.

마무리: 코드 읽는 순서

프로시저와 제어문은 "흐름"을 따라 읽으면 어렵지 않습니다.



- IF는 참/거짓 분기, CASE는 여러 조건 중 하나를 고르는 구조입니다.
- WHILE은 조건이 참일 때 계속 반복하므로, 반복 조건이 바뀌는 코드가 필요합니다.
- ITERATE는 이번 반복을 건너뛰고, LEAVE는 반복문을 종료합니다.
- PREPARE → EXECUTE → DEALLOCATE 순서로 동적 SQL을 실행합니다.
- 문자열 결합은 CONCAT(), 날짜 차이는 DATEDIFF()를 사용합니다.

질문 예시: “4의 배수를 제외하는 위치를 바꾸면 결과가 어떻게 달라질까?”